

Practical Live-Coding Challenge

Auto-Search Input with Debounce & Race Condition Guard
Problem Statement Build an auto-search component that:

- Fetches data from an API based on user input.
- Debounces the user input by 500ms to avoid unnecessary API calls on every keystroke.
- Prevents race conditions if a slow network request resolves after a newer, faster request.
- Handles loading, error, and empty states gracefully.

Solution Implementation

```
import React, { useState, useEffect } from 'react';

// Custom hook to debounce fast-changing values
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup timer if value or delay changes before completion
    return () => clearTimeout(handler);
  }, [value, delay]);

  return debouncedValue;
}

export default function SearchUsers() {
  const [searchTerm, setSearchTerm] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Debounce search term by 500ms
  const debouncedSearchTerm = useDebounce(searchTerm, 500);

  useEffect(() => {
    // Skip empty searches
    if (!debouncedSearchTerm.trim()) {
      setResults([]);
      setLoading(false);
      return;
    }

    // Flag to prevent race conditions
    let isCurrent = true;
    setLoading(true);
    setError(null);
```

```

async function fetchUsers() {
  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/users?q=${encodeURIComponent(
        debouncedSearchTerm
      )}`
    );
    if (!response.ok) throw new Error('Network error');

    const data = await response.json();

    // Update state only if this request is still the latest one
    if (isCurrent) {
      setResults(data);
    }
  } catch (err) {
    if (isCurrent) {
      setError(err.message || 'Something went wrong');
    }
  } finally {
    if (isCurrent) {
      setLoading(false);
    }
  }
}

fetchUsers();

// Cleanup: invalidate the request if component unmounts or searchTerm changes
return () => {
  isCurrent = false;
};
}, [debouncedSearchTerm]);

return (
  <div style={{ padding: '20px', maxWidth: '400px' }}>
    <h2>User Search</h2>
    <input
      type="text"
      placeholder="Type a name to search..."
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
    />

    {loading && <p>Loading results...</p>}
    {error && <p style={{ color: 'red' }}>Error: {error}</p>}}

    {!loading && !error && results.length === 0 && debouncedSearchTerm && (
      <p>No results found.</p>
    )}
  )

```

```
    <ul>
      {results.map((user) => (
        <li key={user.id}>{user.name} ({user.email})</li>
      ))}
    </ul>
  </div>
);
}
```

4 Frequently Asked Follow-Up Topics

Concept	Key Point Evaluated	Common Pitfalls to Avoid
State Updating	Functional updates (<code>setCount(prev => prev + 1)</code>)	Relying on stale state variables directly inside async handlers or timers.
Memoization	<code>React.memo</code> , <code>useCallback</code> , <code>useMemo</code> usage	Re-creating inline functions/objects passed as props to memoized children.
Effect Cleanup	Returning cleanup functions in <code>useEffect</code>	Forgetting to clear intervals, timeouts, or window event listeners.
List Rendering	Using unique, stable IDs as key props	Using array index as key when list order can change.

- 1) **InterviewsNested Comments Component:** Build a multi-level threaded comment UI using recursive React components.
- 2) **Transfer List:** Create two side-by-side lists with checkboxes allowing items to be transferred back and forth.
- 3) **Custom useFetch Hook:** Implement a custom hook handling loading, error states, and automatic request cancellation via `AbortController`.
- 4) **Paginated Data Table:** Build a sortable data table fetching paginated data from an API endpoint.